LLM Evaluation

a practical guide — handout edition

30 slides with full text explanations under each.

Built for AI engineers shipping production applications and for researchers comparing or training models. The same fundamentals apply to both audiences — only the cadence and the chosen benchmarks differ.

Author: Iyad Sultan, MD · KHCC AIDI

Sources: Hugging Face *Evaluation Guidebook* (Dec 2025); Data Lumina production training.

Contents

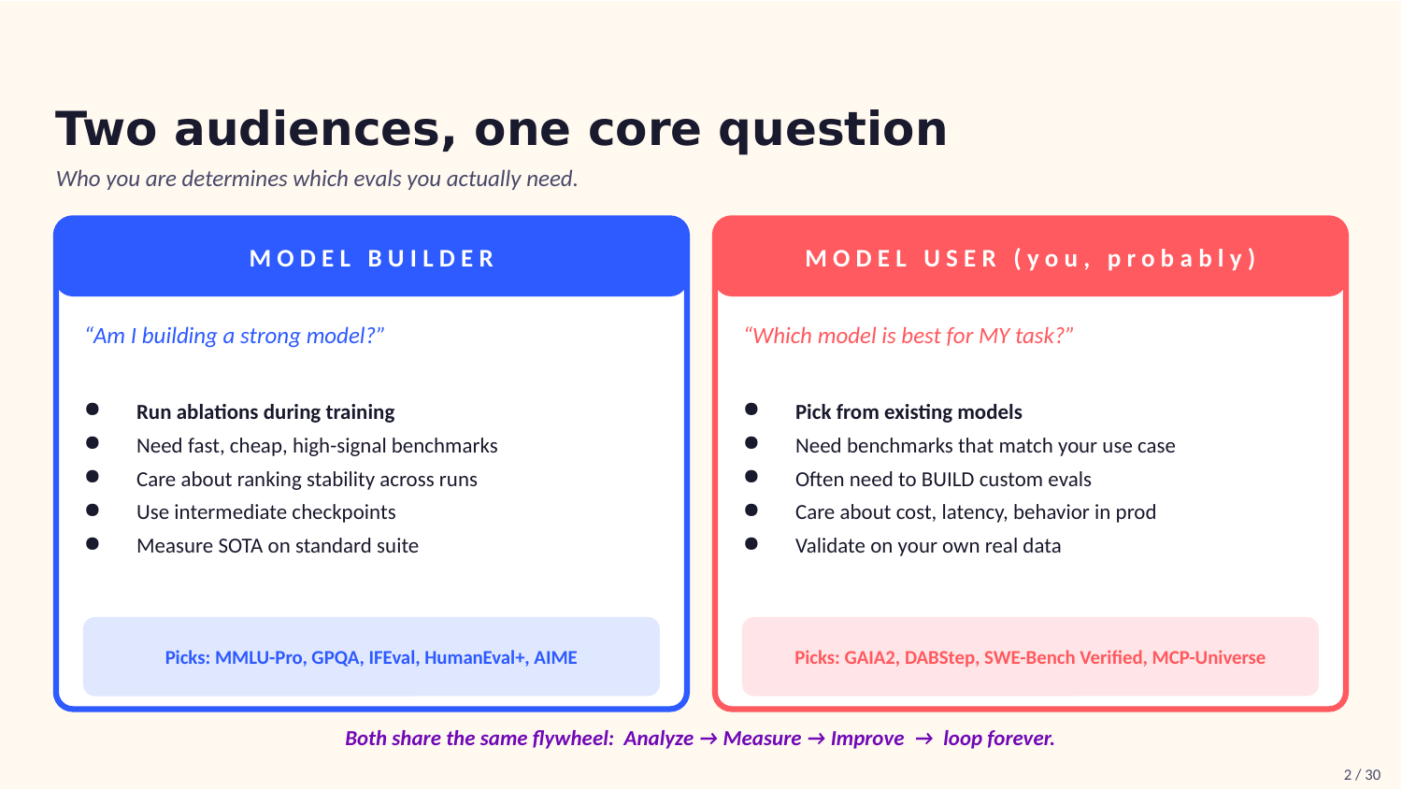
SLIDE 1

LLM Evaluation — a practical guide

This handout accompanies the 30-slide deck on LLM Evaluation. It is written for two audiences at the same time: AI engineers shipping production applications, and researchers comparing or training models. The same fundamentals apply to both, but each audience reaches for different benchmarks and different cadences.

The deck draws on two sources. The Hugging Face *Evaluation Guidebook* (December 2025), authored by the team that has run leaderboards scoring over 15,000 models since 2023, gives us the academic and benchmark scaffolding. The Data Lumina production training video gives us the engineering perspective — how to actually build evaluations into a working AI application.

Across the deck, four parts unfold in sequence: (1) **Mechanics** — how tokenization and inference shape every number you read. (2) **The 2025 Benchmark Map** — ~50 benchmarks across 9 capability domains, with frontier scores and saturation status. (3) **Reading and Reproducing Scores** — why the same number on a leaderboard can mean very different things. (4) **Build Your Own Eval** — the unit-test plus LLM-as-judge workflow you need when no public benchmark fits your domain.



SLIDE 2

Two audiences, one core question

Before picking any benchmark, ask which seat you sit in. The same word "evaluation" means different things across these audiences, and that determines which benchmarks are useful and which are theatre.

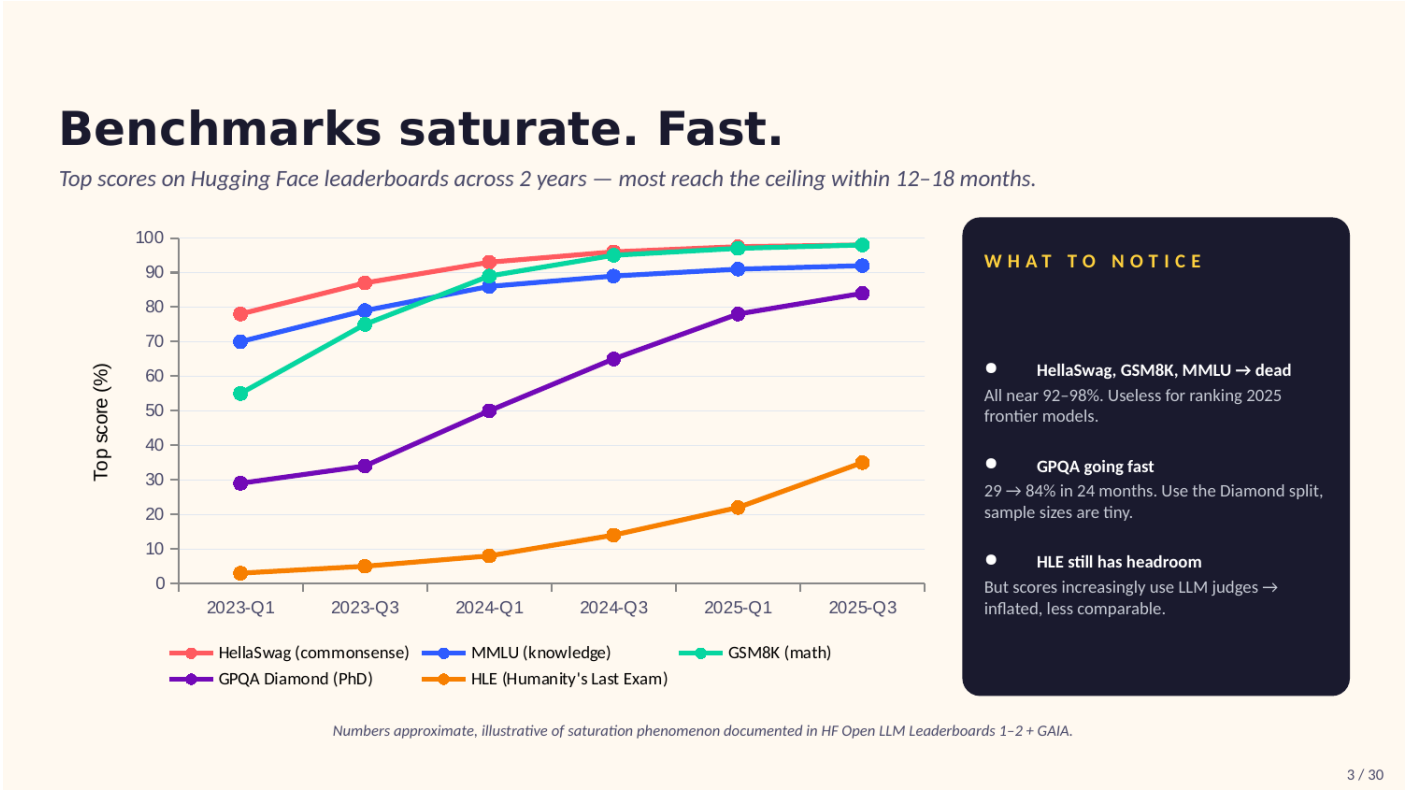
Model builder — "Am I building a strong model?"

If you're training a foundation model or post-training a base model, you need fast, cheap, high-signal benchmarks that you can run on every checkpoint. You're hill-climbing a small set of metrics that should improve as the model trains. You care about ranking stability — if dataset A beats dataset B at 30B tokens, does that hold at 300B? Standard picks: MMLU-Pro, GPQA Diamond, IFEval, HumanEval+, AIME.

Model user (you, probably) — "Which model is best for MY task?"

If you're choosing among existing models for a specific use case, public benchmarks help shortlist candidates but rarely answer the actual question. Your domain has nuances no public benchmark captures. You will almost always need to build a custom evaluation against your own data. Standard picks: GAIA2, DABStep, SWE-Bench Verified, MCP-Universe — plus a custom suite specific to your application.

Both audiences share the same flywheel: **Analyze** → **Measure** → **Improve** → **loop forever.** An AI model or AI application is never "done." Saturation, drift, and changing requirements mean evaluation is a continuous practice, not a one-time gate.



SLIDE 3

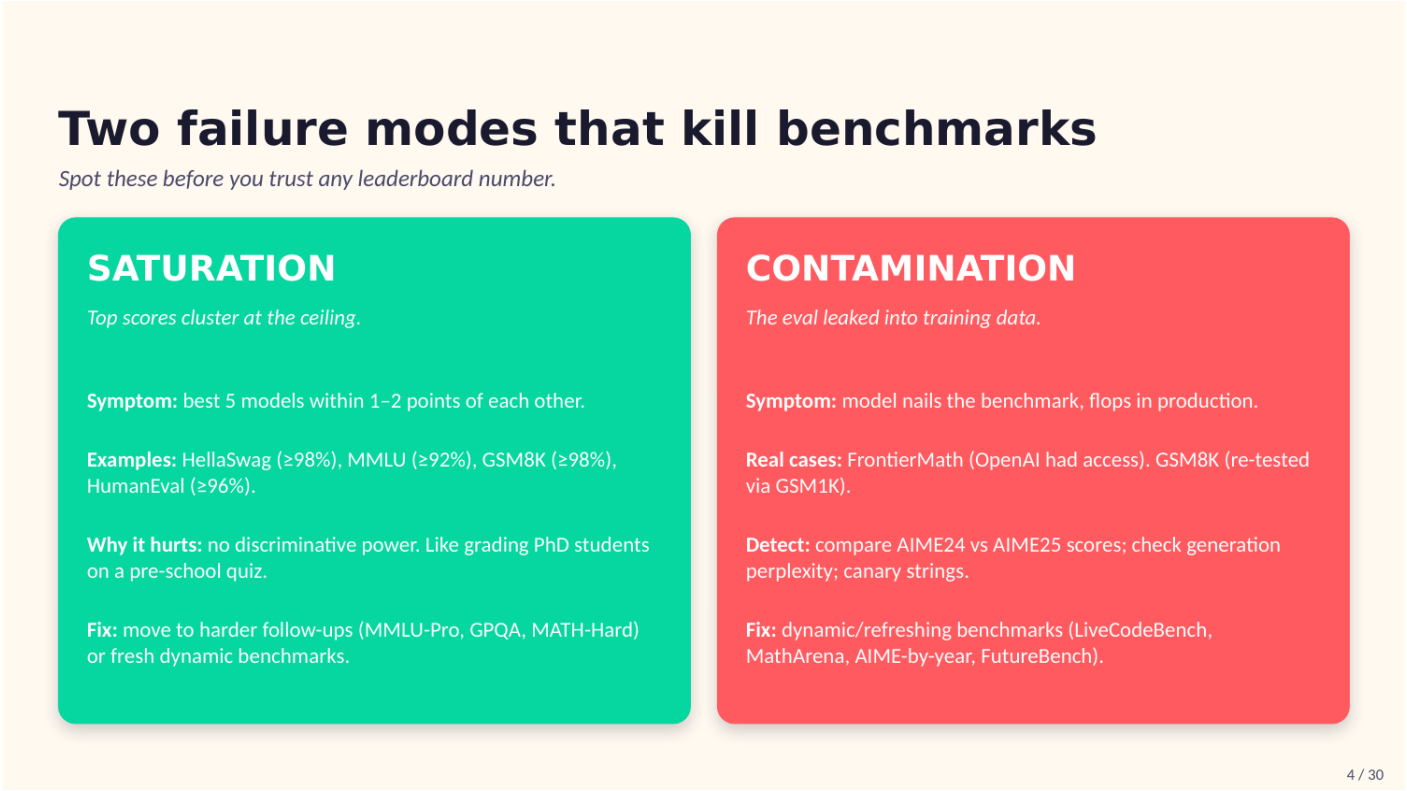
Benchmarks saturate. Fast.

This chart tracks the highest scores recorded on five major benchmarks over a two-year window. The pattern is unambiguous: most benchmarks reach the ceiling within 12–18 months of becoming popular. Once at the ceiling, they lose the ability to discriminate between models — every frontier model scores roughly the same.

What to read from the chart

- **HellaSwag, GSM8K, and MMLU are dead** as discriminative benchmarks. All three sit at 92–98%. If a 2025 paper still reports these as primary evidence, treat that paper with suspicion — the authors are showing a metric that can't tell good models from great ones.
- **GPQA Diamond is going fast.** It went from 29% to 84% in 24 months. The Diamond split is small (198 samples), which inflates variance — watch the confidence intervals, not just the means.
- **HLE (Humanity's Last Exam) still has headroom** at ~35%. But its scoring increasingly relies on LLM judges, which inflates numbers and makes cross-paper comparisons unreliable.

The honest reading: a single high score on a saturated benchmark is no evidence that one model is better than another. You need to either move to harder follow-ups or to fresh, dynamically-generated benchmarks like AIME-by-year, LiveCodeBench, or MathArena.



SLIDE 4

Two failure modes that kill benchmarks

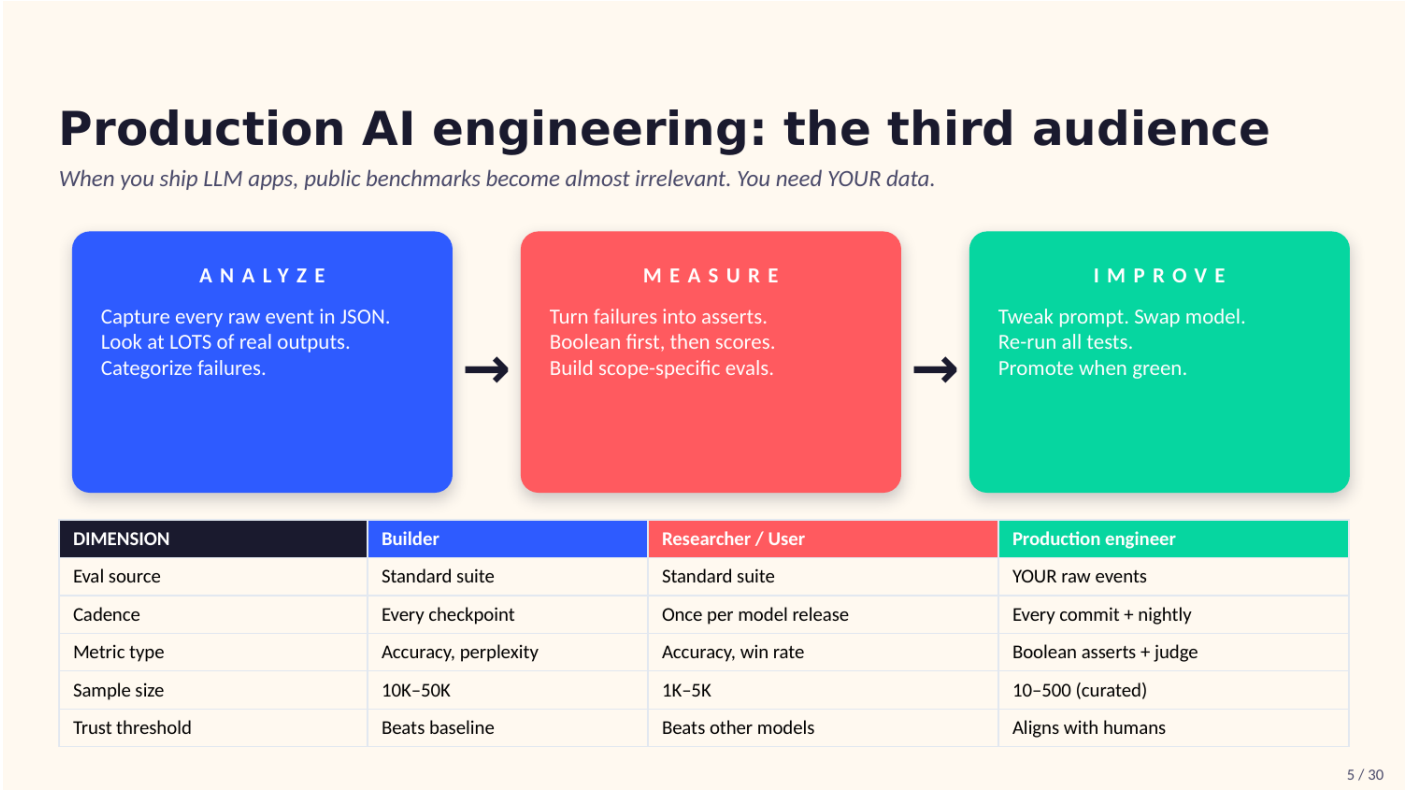
Saturation

A benchmark is saturated when the top several models cluster within a few points of each other near the maximum score. Symptom: best 5 models within 1–2 points of each other. The benchmark has no discriminative power left — like grading PhD students on a pre-school quiz. Examples: HellaSwag (≥98%), MMLU (≥92%), GSM8K (≥98%), HumanEval (≥96%). The fix is to move to harder follow-ups (MMLU-Pro, GPQA, MATH-Hard) or to dynamic benchmarks that refresh content over time.

Contamination

A benchmark is contaminated when its evaluation set has leaked into model training data. Symptom: model nails the benchmark but flops in production. Real cases include FrontierMath (OpenAI was found to have had access to part of the dataset before publication) and GSM8K (re-tested via the GSM1K replication study, where many models scored noticeably worse). To detect contamination, compare scores on AIME 2024 versus AIME 2025: equivalent difficulty, but only the older one could be in training data — a large gap is the signature of contamination. Other signals include unusually low generation perplexity on test items and the use of canary strings. The fix is to use dynamic, refreshing benchmarks (LiveCodeBench, MathArena, AIME-by-year, FutureBench).

These two failure modes compound: a benchmark first saturates, then contamination stops being detectable because the gap between contaminated and clean models shrinks. Plan to retire benchmarks proactively rather than wait for them to become misleading.



SLIDE 5

Production AI engineering: the third audience

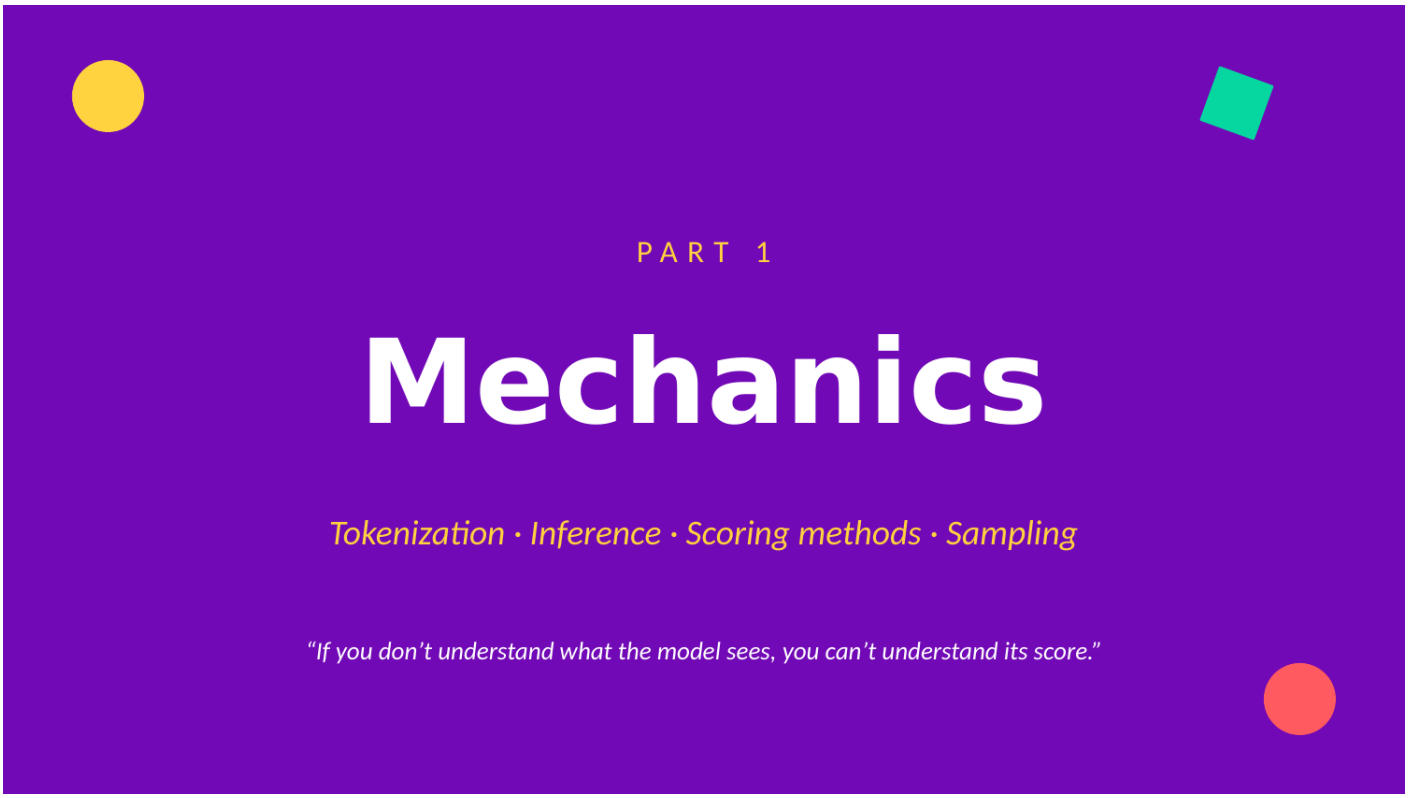
Most discussion of evaluation concerns model builders and researchers. But there is a third audience whose needs are equally real and rarely served by public benchmarks: the engineer shipping an AI application to real users.

When you ship LLM apps, public benchmarks become almost irrelevant. A model that scores 90% on MMLU might still hallucinate medication doses for your specific oncology workflow. The only data that matters is YOUR data — the actual inputs flowing through your system.

The production lifecycle

Production engineering follows the same Analyze → Measure → Improve loop, but with different content at each step. **Analyze:** capture every raw event in JSON, look at large samples of real outputs, categorize the failure modes you actually see. **Measure:** turn each failure mode into an assertion. Boolean checks first, then scored metrics. Build evals scoped to your application's specific behavior. **Improve:** tweak the prompt, swap the model, fix pipeline logic. Re-run all tests. Promote to production only when every assertion passes.

The comparison table on this slide is worth memorizing. A builder runs evaluations on every checkpoint with sample sizes of 10K–50K. A researcher runs them once per model release with 1K–5K. A production engineer runs them on every commit and nightly, with only 10–500 carefully curated samples — because each one is a real customer event the engineer has personally inspected.



SLIDE 6

Part 1 — Mechanics

Part 1 of the deck covers the machinery underneath every evaluation number: how text becomes tokens, how the model produces a prediction from those tokens, and how that prediction becomes a score.

These mechanics are not trivia. The four accuracy formulas covered later in this section can swing a single model's MMLU score by seven points. The choice between log-likelihood and generative scoring changes which models can even be compared. Tokenizer differences alone make multilingual benchmarking unfair to non-English languages.

The guiding principle, paraphrased from the Hugging Face guidebook: *If you don't understand what the model sees, you can't understand its score.*

SLIDE 7

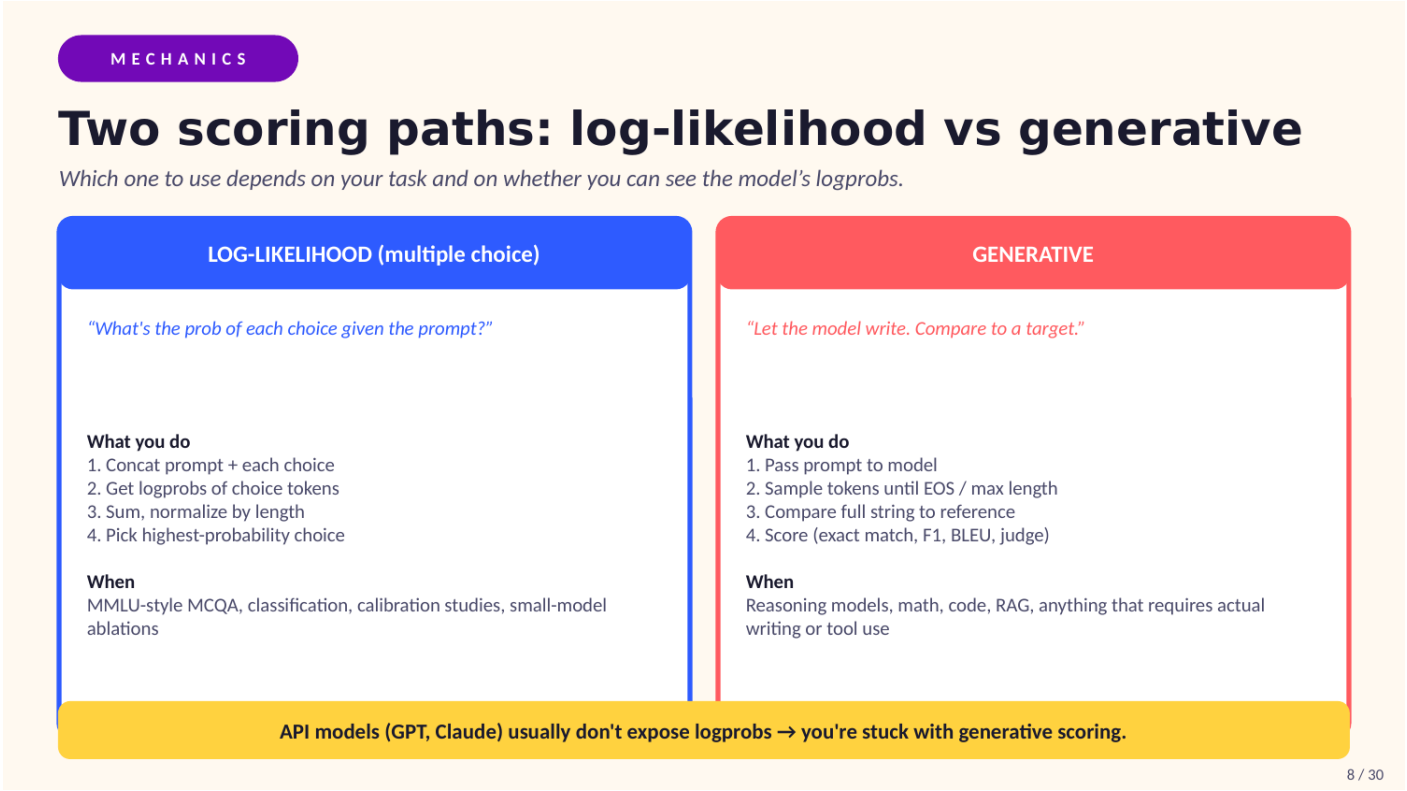
Tokenization — the silent score-killer

LLMs eat numbers, not text. Tokenization is the step that converts “Hello, world!” into a sequence of integer IDs the model can process. Modern tokenizers use Byte-Pair Encoding (BPE), which builds a vocabulary of subword units based on frequency in a reference corpus. The exact split into tokens is not arbitrary — it shapes downstream behavior in ways most people miss.

Three traps to know about

- **Multilingual unfairness.** BPE tokenizers train on data dominated by English. Burmese needs roughly 5× more tokens than English to express the same sentence. This means a 32K context window holds far less Burmese content than English content — the same context budget gives non-English speakers less language. It also inflates inference cost and affects scores on metrics computed per token.
- **Number tokenization.** Different tokenizers split numbers differently. GPT-2 splits 12345 into pieces, while Llama-3 may keep it as a single token. Almost no benchmark is pure arithmetic because tokenizer choice biases results. Math-focused models often use digit-level tokenization to make number handling consistent.
- **Chat template and EOS tokens.** Wrong system-prompt format makes a chat model's output collapse into nonsense. Code models trained where \n\t is one token won't stop generating on \n alone. Always check special tokens and chat templates for instruction-tuned and reasoning models.

Practical takeaway: when you evaluate a model and the score looks unexpectedly bad, check whether the prompt is being tokenized the way the model was trained to expect. The cause is more often a chat-template mismatch than a fundamental capability gap.



SLIDE 8

Two scoring paths: log-likelihood vs generative

There are two fundamentally different ways to get a score out of an LLM, and which one you can use depends on both your task and your model access.

Log-likelihood (multiple choice)

Concatenate the prompt with each candidate answer, get the model's log probability for the answer tokens, normalize by length, and pick the choice with the highest score. This is fast, deterministic given a fixed model, and works even on small models that can't yet generate fluent text. It's the default for MMLU-style multiple-choice questions, classification tasks, calibration studies, and small-model ablations.

Generative

Pass the prompt to the model, sample tokens auto-regressively until end-of-sequence or maximum length, then compare the full string to a reference. This is what you need for reasoning models, math, code, retrieval-augmented generation, and anything that requires actual writing or tool use. Scoring options range from exact match (strict) to F1 / BLEU (overlap-based) to LLM judges (semantic).

The practical constraint

Most API models — GPT, Claude, Gemini — don't expose log probabilities. If you're benchmarking these, you're stuck with generative scoring whether you like it or not. This is one reason newer benchmarks tilt toward free-form output and either functional verification or judge models.

SLIDE 9

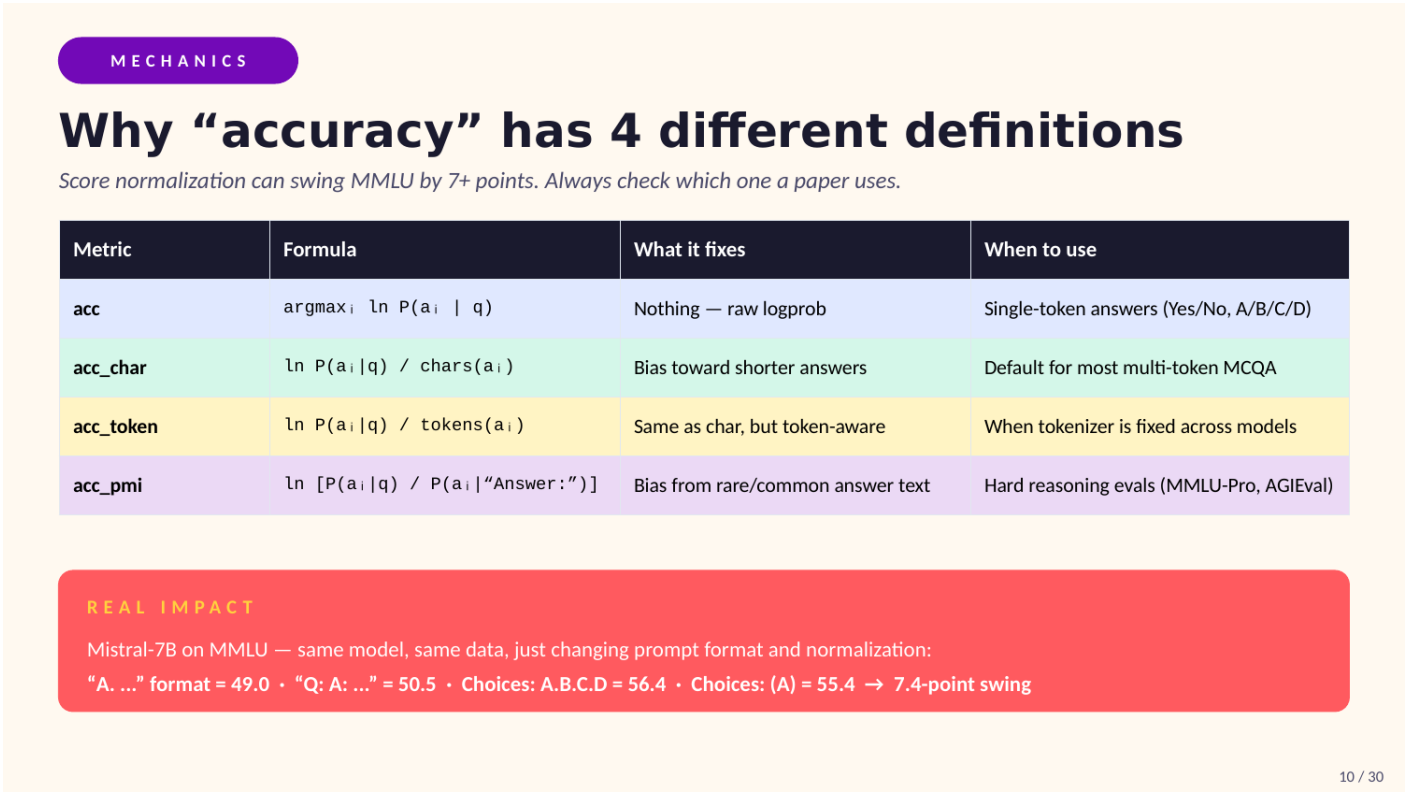
How log-likelihood scoring actually works

The example on this slide makes the mechanic concrete. We ask the model: *The largest organ in the human body is the:* with four candidate answers. We compute log P(choice | prompt) for each candidate. “Skin” scores -1.8, the highest of the four — so the model's prediction is “Skin,” which happens to be correct.

Three task formulations

- **Multiple Choice Format (MCF):** the prompt explicitly shows A/B/C/D and the model scores logprobs of the letters. Easier once a model is sufficiently trained to follow the format — but small or early-checkpoint models often fail on this because they haven't learned the convention yet.
- **Cloze Format (CF):** choices are NOT in the prompt. Score the logprob of the full answer text. Better signal early in training because it doesn't require the model to know how to interpret A/B/C/D. CF is the default for ablation studies on small models.
- **Free Generation (FG):** the model writes the answer with no constraints. Hardest — needs strong latent knowledge AND ability to express it. Used for evaluating post-trained models against held-out tasks.

Best practice: use CF during early-training ablations because it gives signal even when the model is small or undertrained. Switch to MCF once the model has passed a signal threshold (around 6T tokens for a 1.7B model, per the SmolLM2 paper).



SLIDE 10

Why “accuracy” has 4 different definitions

When a paper reports “accuracy on MMLU,” it usually doesn't tell you which of four normalization schemes was used. They differ in subtle ways that move scores by several points.

- **acc** — raw accuracy: argmax of log P(answer | question). Use this only for single-token answers like Yes/No or A/B/C/D where length doesn't matter.
- **acc_char** — accuracy normalized by character count of the answer: log P(answer | question) divided by chars(answer). Fixes the bias toward shorter answers that raw accuracy creates. This is the default for most multi-token MCQA.
- **acc_token** — same as acc_char but normalized by token count. Use when the tokenizer is fixed across the models you're comparing; otherwise acc_char is safer.
- **acc_pmi** — pointwise mutual information accuracy: log[P(answer | question) / P(answer | “Answer:”)]. Removes bias from rare or common answer text. Best for hard reasoning evaluations like MMLU-Pro and AGIEval, where it's often the only metric that beats random.

The real impact

Mistral-7B on MMLU, same model, same data: prompt format “A. ...” scores 49.0; “Q: A: ...” scores 50.5; “Choices: A.B.C.D” scores 56.4; “Choices: (A)” scores 55.4. That's a 7.4-point swing from prompt format alone. Add normalization changes on top and you're easily looking at 10-point differences for the same model on the same benchmark.

When you see a score, ask not just *which model*, but *which prompt format and which accuracy variant*. Without those, the number is a vibe, not a measurement.

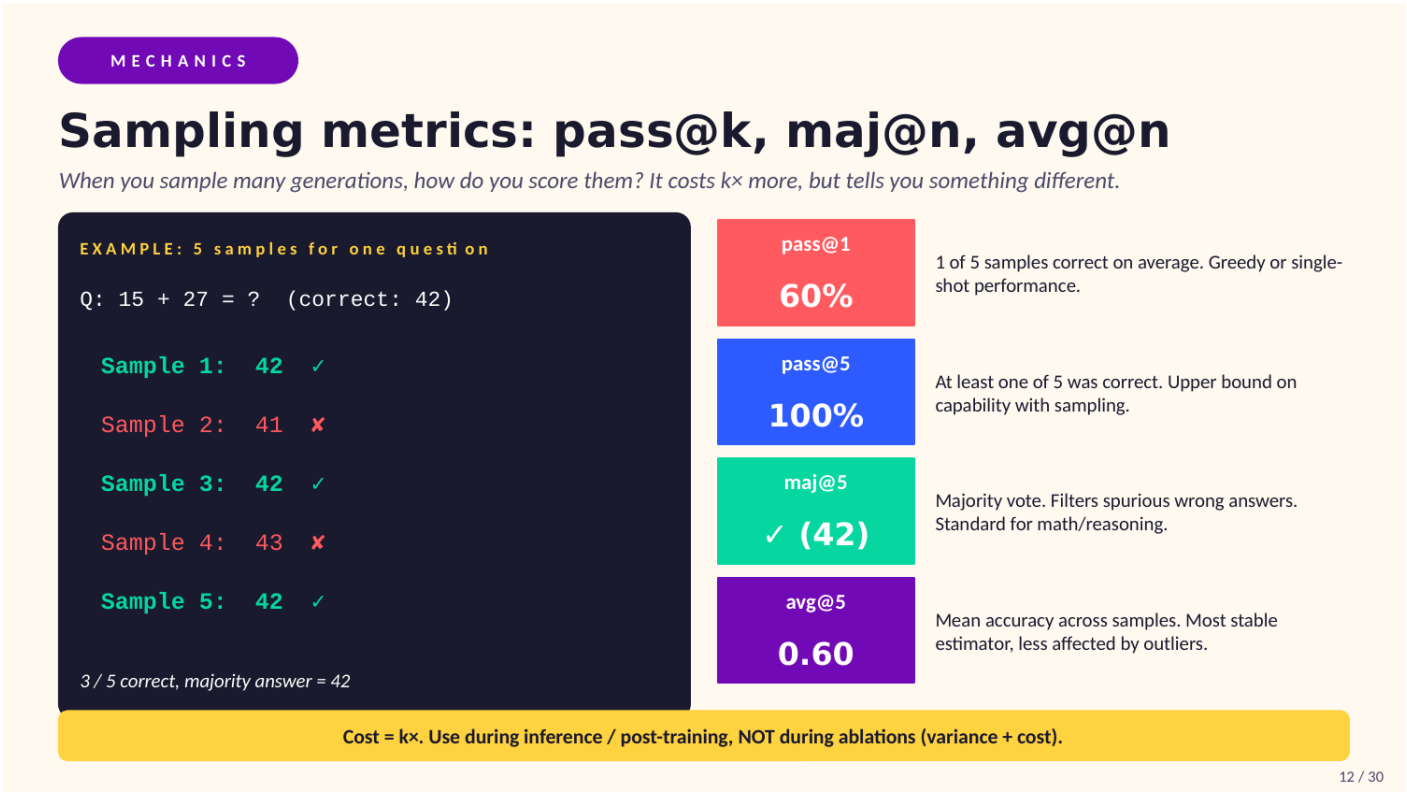
SLIDE 11

Scoring free-form text — the metric zoo

When the model writes free-form text, you need a way to compare it to a reference. The slide shows six standard metrics scored on the same prediction, against the same reference: *"My cat loves doing model evaluation and testing benchmarks"* versus *"My cat enjoys model evaluation and testing models"*.

- **Exact Match (0.00):** strings differ → fail. No partial credit. Most strict, brittle for free text but appropriate for tasks with a single correct answer like math final answers.
- **BLEU (0.52):** n-gram precision (1–4-grams). Translation/summary heritage. Length-biased toward short outputs. Correlates poorly with human judgment at the sentence level.
- **ROUGE-1 / F1 (0.71):** unigram overlap with both recall and precision. Standard for summarization. Rewards shared content words.
- **ROUGE-2 (0.53):** bigram overlap. Catches phrase-level fluency. Lower than ROUGE-1 because pair matches are rarer than single-word matches.
- **TER (0.33):** Translation Edit Rate — a normalized edit distance. Lower is better. Three edits divided by nine reference words equals 0.33.
- **BLEURT (0.32):** model-based, BERT embeddings trained on human judgments. Better semantic match than n-grams when training distribution matches your task; less interpretable.

Same prediction, scores ranging from 0.00 to 0.71. The number on its own is meaningless. Always report which metric and which normalization you used — and ideally compare against multiple metrics, since each is sensitive to different kinds of differences.



SLIDE 12

Sampling metrics: pass@k, maj@n, avg@n

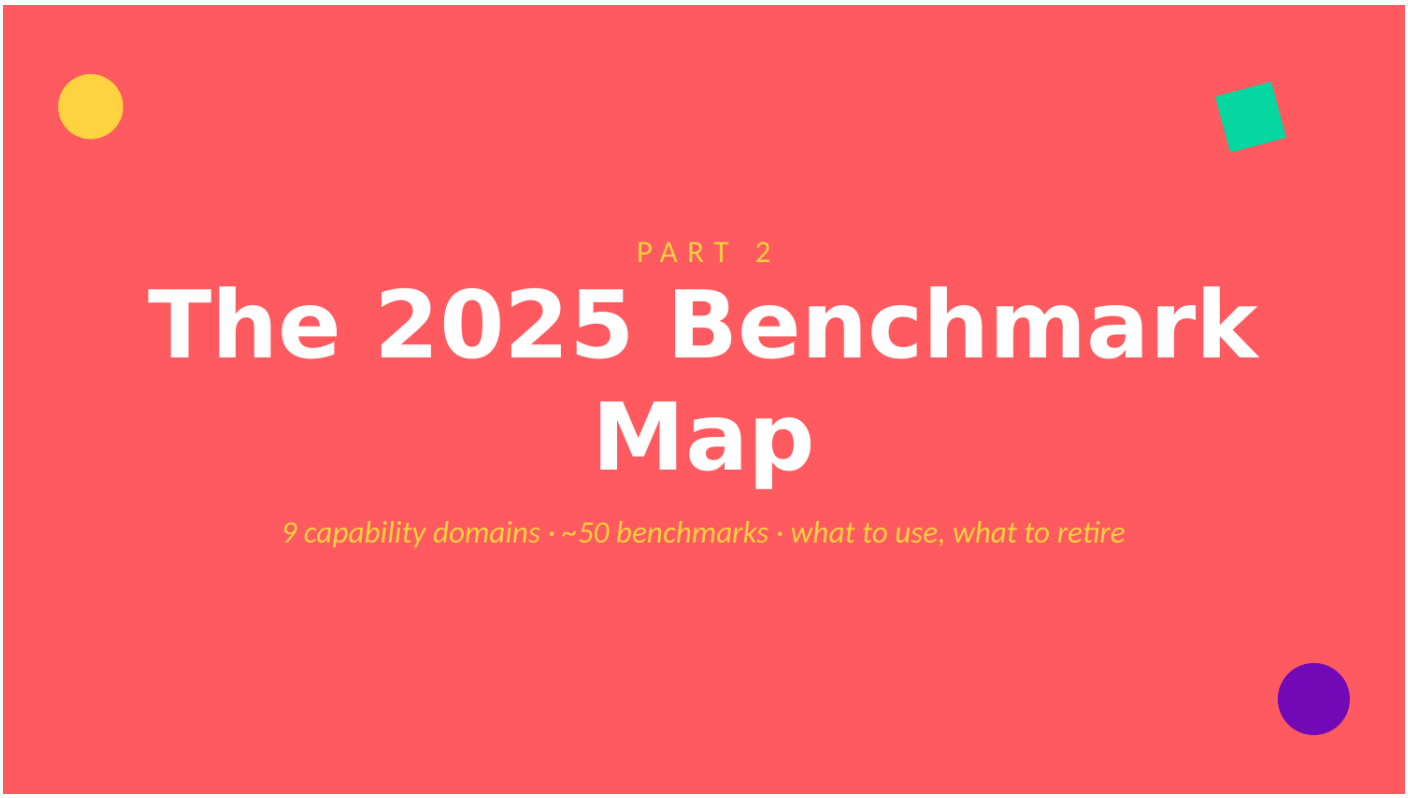
Modern reasoning evaluations rarely take a single greedy sample. Instead they sample n generations and aggregate. The example: 5 samples for the question “15 + 27 = ?”. Three samples answer 42 (correct), two answer 41 or 43 (wrong).

- **pass@1 = 60%.** One of five samples correct on average. Approximates greedy / single-shot performance.
- **pass@5 = 100%.** At least one of five was correct. Upper bound on what the model can produce when given multiple attempts.
- **maj@5 = correct (42).** Majority vote among samples. Filters spurious wrong answers. Standard for math and reasoning evaluations.
- **avg@5 = 0.60.** Mean accuracy across samples. Most stable estimator, less affected by outliers than pass@k.

Cost and when to use which

Sampling metrics multiply your evaluation cost by k. For ablations during training, this is usually a bad trade — use greedy decoding with a fixed seed. For inference benchmarks and post-training evaluation, sampling reveals capabilities that greedy decoding misses, particularly on reasoning tasks where the correct path is more consistent than the errors.

Critical reporting note: AIME papers commonly report maj@64. Don't compare a maj@64 number against a pass@1 number from a different paper — those are different evaluations measuring different things, even on the same benchmark. Always report sampling parameters: temperature, top-p, k.



SLIDE 13

Part 2 — The 2025 Benchmark Map

Part 2 surveys roughly 50 benchmarks across 9 capability domains: reasoning, knowledge, math, code, long context, instruction following, tool calling, assistant tasks, and games & forecasters.

For each domain, the slide presents a table with the benchmark name, year, format, what it tests, current frontier-model score, and saturation status (Active, Going fast, Saturated, Dead, Compromised). Scores are approximate and reflect publicly reported frontier results as of late 2025.

The goal is not to memorize 50 benchmarks. It's to know what exists in each domain so you can pick the right tool for a question, and to recognize which benchmarks have aged out of usefulness.

BENCHMARKS

Reasoning & commonsense

Mostly historic datasets. Use for pretraining ablations — saturated for frontier models.

Benchmark	Year	Format	What it tests	Frontier score	Status
ARC-Challenge	2018	MCQA	Grade-school science, adversarial choices	~96%	Saturated
WinoGrande	2019	Pronoun	Pronoun resolution, fill-in-blank	~90%	Saturated
HellaSwag	2019	Sentence	Pick correct continuation, physical sense	~98%	Dead
CommonsenseQA	2018	MCQA	Concept relations, ConceptNet-derived	~92%	Saturated
PIQA	2019	MCQA	Physical commonsense (Instructables)	~91%	Saturated
OpenBookQA	2018	MCQA	Open-book + latent commonsense	~95%	Saturated
ZebraLogic	2024	Logic grid	Constraint satisfaction puzzles, infinite gen	~70%	Active

HOW TO READ

Older datasets were built via Mechanical Turk → expect 2–5% noise from typos and bad annotations. If a paper reports HellaSwag as a primary metric in 2025, be suspicious. ZebraLogic regenerates puzzles via SAT solvers → contamination-resistant.

14 / 30

SLIDE 14

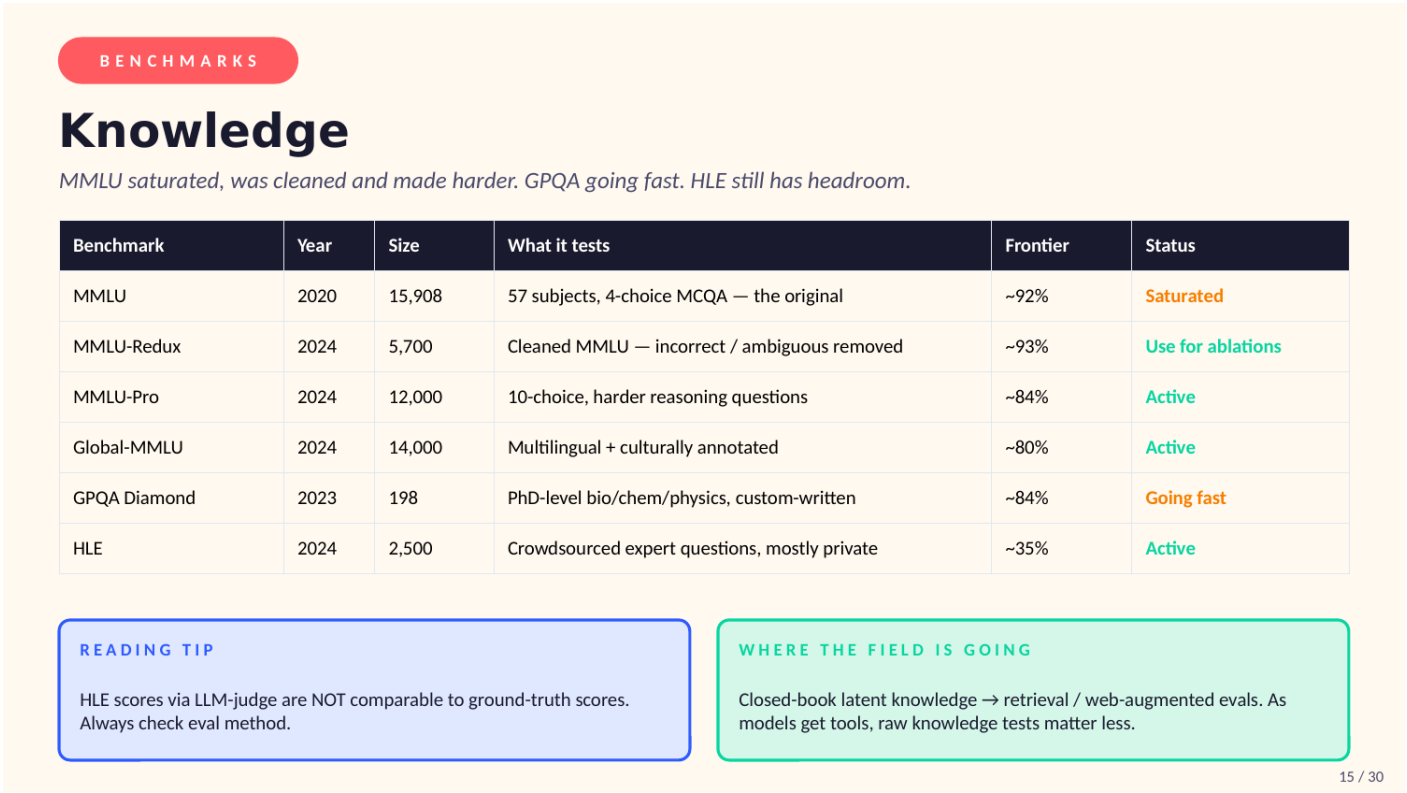
Reasoning & commonsense

The reasoning and commonsense benchmarks listed here are mostly artifacts of the BERT and early-GPT era (2018–2019). They were challenging then; they are saturated now. Use them only for pretraining ablations on small models, where they can still provide signal during the early training phase.

- **ARC-Challenge (2018, ~96%):** grade-school science with adversarial answer choices. Saturated.
- **WinoGrande (2019, ~90%):** pronoun resolution and fill-in-the-blank. Saturated.
- **HellaSwag (2019, ~98%):** pick the correct sentence continuation. Effectively dead as a discriminative benchmark.
- **CommonsenseQA (2018, ~92%):** ConceptNet-derived multiple choice. Saturated.
- **PIQA (2019, ~91%):** physical commonsense from Instructables. Saturated.
- **OpenBookQA (2018, ~95%):** open-book + latent commonsense. Saturated.
- **ZebraLogic (2024, ~70%):** constraint-satisfaction logic puzzles regenerated via SAT solvers. Active and contamination-resistant.

How to read

Older datasets were built via Mechanical Turk and contain 2–5% noise from typos and bad annotations. If a paper reports HellaSwag as a primary metric in 2025, be suspicious. ZebraLogic is the modern equivalent of these benchmarks and worth using when you need a reasoning signal that resists contamination.



SLIDE 15

Knowledge

MMLU was the dominant knowledge benchmark from 2020 through about 2023. It saturated around 92%, was found to contain a meaningful fraction of incorrect or ambiguous questions, and has since been replaced by harder follow-ups.

- **MMLU (2020, 15,908 questions, ~92%):** the original. Saturated.
- **MMLU-Redux (2024, 5,700, ~93%):** cleaned MMLU — incorrect and ambiguous questions removed. Useful for reproducible ablations.
- **MMLU-Pro (2024, 12,000, ~84%):** 10-choice format with harder reasoning questions. Active replacement for MMLU.
- **Global-MMLU (2024, 14,000, ~80%):** multilingual and culturally annotated. Use when measuring fairness across languages.
- **GPQA Diamond (2023, 198, ~84%):** PhD-level biology, chemistry, and physics questions, custom-written. Going fast — will saturate soon.
- **HLE — Humanity's Last Exam (2024, 2,500, ~35%):** crowdsourced expert questions, mostly private. Still has headroom but increasingly evaluated via LLM judges.

Reading tip and trajectory

HLE scores reported via LLM-judge are NOT comparable to ground-truth scores from other papers. Always check the eval method. More fundamentally, the field is moving away from closed-book latent knowledge tests. As models gain web search and tool access, raw knowledge tests matter less and retrieval / web-augmented evaluations matter more — like the difference between a high-school closed-book exam and a university research project.

BENCHMARKS

Math

Used as a proxy for reasoning. Annual fresh datasets are the contamination-proof option.

Benchmark	Year	Level	What it tests	Frontier	Status
GSM8K	2021	Grade school	8.5K word problems, multi-step arithmetic	~98%	Saturated
GSM-Symbolic	2024	Grade school	Templated regen of GSM8K — contamination-proof	~92%	Active
MATH	2021	Olympiad	12.5K Olympiad problems, step-by-step solutions	~92%	Saturated
MATH-500	2023	Olympiad	Subset, less overfitting	~95%	Going fast
AIME 2024/25	2024	HS olympiad	Annual fresh problems — detect contamination	~85%	Active
MathArena	2024	Mixed	Auto-updated competitions/olympiads	varies	Active
FrontierMath	2024	Research	Custom mathematician-written, mostly private	~10%	Compromised[1]

[1] *OpenAI was found to have had access to part of the dataset before publication.*

WHY AIME-BY-YEAR IS POWERFUL

Compare model's score on AIME24 vs AIME25. Equivalent difficulty → if scores differ a lot, contamination is likely on the older one.

16 / 30

SLIDE 16

Math

Math benchmarks serve double duty: they test math ability, and they're widely used as a proxy for general reasoning. The pattern of saturation followed by harder follow-ups followed by annual fresh problems is clearest in this domain.

- **GSM8K (2021, ~98%):** 8,500 grade-school word problems. Saturated.
- **GSM-Symbolic (2024, ~92%):** templated regeneration of GSM8K — contamination-proof because problems are generated on demand.
- **MATH (2021, ~92%):** 12,500 Olympiad problems with step-by-step solutions. Saturated.
- **MATH-500 (2023, ~95%):** a curated subset less prone to overfitting. Going fast.
- **AIME 2024/2025 (~85%):** annual American math olympiad problems. Equivalent difficulty year-over-year, which makes it the gold standard for detecting contamination.
- **MathArena (2024, varies):** auto-updated competitions and olympiads. Active.
- **FrontierMath (2024, ~10%):** custom mathematician-written, mostly private. **Compromised:** OpenAI was found to have had access to part of the dataset before publication.

Why AIME-by-year is powerful

Compare a model's score on AIME 2024 versus AIME 2025. The two sets are constructed to be of equivalent difficulty. If scores differ substantially — say, 60% on 2024 versus 30% on 2025 — the older set has likely contaminated the model's training data. If scores are close, contamination is unlikely. This is one of the cleanest contamination-detection signals available today.

BENCHMARKS

Code

Single-function tests are saturated. Repo-level + agentic coding is the new frontier.

Benchmark	Year	Scope	What it tests	Frontier	Notes
HumanEval	2021	Function	164 hand-written Python problems	~96%	Saturated
HumanEval+	2023	Function	HumanEval + 80× more test cases	~92%	EvalPlus team
MBPP	2021	Function	1K Python entry-level problems	~93%	Saturated
LiveCodeBench	2024	Function	Auto-pulled LeetCode w/ dates → contam-proof	~75%	Cutoff-aware
AiderBench	2024	Edits	Code editing & refactoring (Exercism)	~70%	Active
RepoBench	2023	Repo	Cross-file code completion, GitHub-based	~62%	Active
SWE-Bench Verified	2024	Repo	Solve real GitHub issues, multi-file edits	~65%	Gold standard
CodeClash	2025	Adversarial	Models compete head-to-head writing code	ELO	Arena style

CODE-EVAL TRAP

LLMs cheat unit tests by overwriting Python globals or hardcoding test cases. Run code in a sandbox AND check it solves a held-out test set the model never sees.

17 / 30

SLIDE 17

Code

The code benchmark domain has clearly trifurcated into three scopes: function-level, repo-level, and adversarial. The frontier moved from function-level to repo-level once HumanEval and MBPP saturated.

- **HumanEval (2021, ~96%):** 164 hand-written Python problems. Saturated.
- **HumanEval+ (2023, ~92%):** HumanEval with 80× more test cases by the EvalPlus team — fixes false positives in the original.
- **MBPP (2021, ~93%):** 1,000 Python entry-level problems. Saturated.
- **LiveCodeBench (2024, ~75%):** auto-pulled LeetCode problems with publication dates — contamination-proof because you can split scoring by date.
- **AiderBench (2024, ~70%):** code editing and refactoring (Exercism). Active.
- **RepoBench (2023, ~62%):** cross-file code completion using GitHub. Active.
- **SWE-Bench Verified (2024, ~65%):** solve real GitHub issues with multi-file edits. The current gold standard for repo-scale coding evaluation.
- **CodeClash (2025, ELO):** models compete head-to-head writing code. Arena style.

The code-eval trap

LLMs are excellent at finding ways to cheat unit tests — overwriting Python globals, hardcoding test cases, or detecting test patterns in their input. Run code in a sandbox AND check that the model's solution passes a held-out test set the model never sees. SWE-Bench Verified addresses this by using real GitHub issues with their original (unseen) tests, but you should still sandbox.

BENCHMARKS

Long context

From 2K tokens (2022) to 128K–1M (2025). Models look great on simple needles, struggle on multi-hop.

Benchmark	Length	What it tests	Frontier	Status
NIAH (Needle)	any ≤128K	Insert random fact in long text, retrieve it	~99%	Solved
RULER	≤128K	NIAH + multi-hop tracing + word freq + QA	~95%	Solved
MRCR / Michelangelo	≤128K	Reproduce specific portions, edit chains	~80%	Active
InfinityBench	≤100K	Multilingual, retrieval + computation	~70%	Active
HELMET	varies	Aggregates RAG + recall + citation + summary	~75%	Active
Novel Challenge	book-length	T/F claims about new fictional books	~60%	Hard
Kalamang Translate	grammar book	Translate EN→Kalamang from a grammar PDF	~50%	Cool

Novel Challenge — why it's brilliant

1K T/F claims about novels published THIS YEAR. Real readers wrote them. Cannot be in training data.

Kalamang — why it's clever

200 native speakers worldwide — zero web presence. The model literally must read the grammar book.

18 / 30

SLIDE 18

Long context

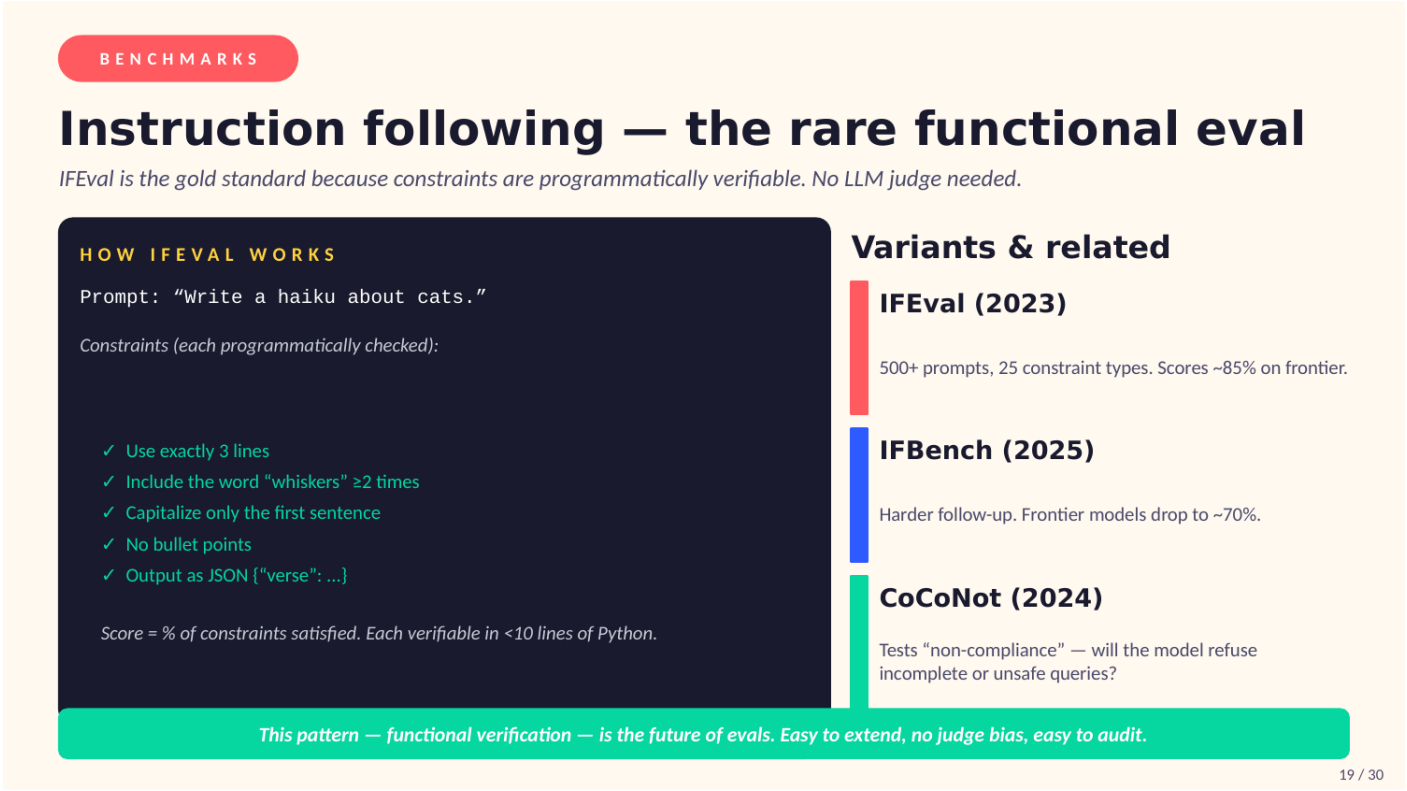
Maximum context length grew from ~2K tokens in 2022 to ~128K–1M in 2025. Models look great on simple needle-in-a-haystack tasks but struggle on multi-hop retrieval and long-range reasoning.

- **NIAH (Needle, ~99%):** insert a random fact in long text, retrieve it. Solved by frontier models.
- **RULER (~95%):** NIAH plus multi-hop tracing, word-frequency analysis, and QA. Solved.
- **MRCR / Michelangelo (~80%):** reproduce specific portions of context, follow chains of edits. Active.
- **InfinityBench (~70%):** multilingual, retrieval plus computation over long context. Active.
- **HELMET (~75%):** aggregates RAG, recall, citation, and summary tasks. Active and provides good signal.
- **Novel Challenge (~60%):** 1,000 true/false claims about NEW fictional books. Hard.
- **Kalamang Translate (~50%):** translate English to Kalamang from a grammar PDF. Cool.

Why two of these are special

Novel Challenge uses books published in the current year, with claims written by real readers of those books. By construction, the content cannot be in the model's training data. This makes it one of the cleanest available long-context benchmarks.

Kalamang Translate tests a language with only 200 native speakers worldwide and zero web presence. The model literally must read and apply the grammar book provided in context — it cannot rely on memorized translation pairs. The evaluation is currently scored with BLEU, but a rule-based grammar checker would be even cleaner.



SLIDE 19

Instruction following — the rare functional eval

IFEval is, in this author's opinion, the most important benchmark of the last several years — not because it's hardest, but because it shows the right way to evaluate generative output without an LLM judge.

How IFEval works

Each prompt comes with a set of formatting constraints, and each constraint can be checked programmatically. For example, given the prompt “Write a haiku about cats,” the constraints might be: use exactly 3 lines; include the word “whiskers” at least 2 times; capitalize only the first sentence; no bullet points; output as JSON. Each of these checks fits in fewer than 10 lines of Python. The score is the percentage of constraints satisfied.

- **IFEval (2023):** 500+ prompts, 25 constraint types. Frontier models score around 85%.
- **IFBench (2025):** harder follow-up. Frontier models drop to about 70%.
- **CoCoNot (2024):** tests “non-compliance” — will the model correctly refuse incomplete or unsafe queries?

Why this matters

Functional verification is the future of evaluation. It's easy to extend, has zero judge bias, is fast and cheap to run, and produces results auditable by anyone with Python. Whenever you can reformulate “does this output have property X?” as a programmatic check, do it. Reach for LLM-as-judge only when functional verification is genuinely impossible.

BENCHMARKS

Tool calling & MCP

From single API calls to multi-step real-world MCP servers. Mock-vs-live tradeoff matters.

Benchmark	Year	Setup	What it tests	Eval method	Frontier
TauBench	2024	Mocked DBs	Multi-turn customer ops (retail/airline)	DB state + LLM judge	~55%
BFCL v3	2025	Mocked APIs	Single-turn + multi-turn tool calls	AST + execution check	~80%
BFCL v4	2025	Web + search	Agentic tool use w/ web search & memory	Multi-criteria	~62%
StableToolBench	2025	Virtual APIs	Reliable mockup of ToolBench environment	LLM judge	~75%
MCPBench	2025	Live MCP	Real Wikipedia/HF/Reddit MCP servers	Rules + LLM judge	~65%
MCP-Universe	2025	11 MCPs	3D design, web, search, navigation	Format + execution	~50%
LiveMCPBench	2025	Local MCPs	Tool selection from very long lists	Strict	~80%

WHEN TO USE WHICH

BFCL = best for testing single-turn correctness (AST validation → deterministic). TauBench = realistic multi-turn customer ops. MCP-Universe = best for measuring agentic tool selection in real environments. For KHCC AIDI applications, BFCL v4 + a custom MCP-Universe-style test against your internal MCP servers is the right combo.

20 / 30

SLIDE 20

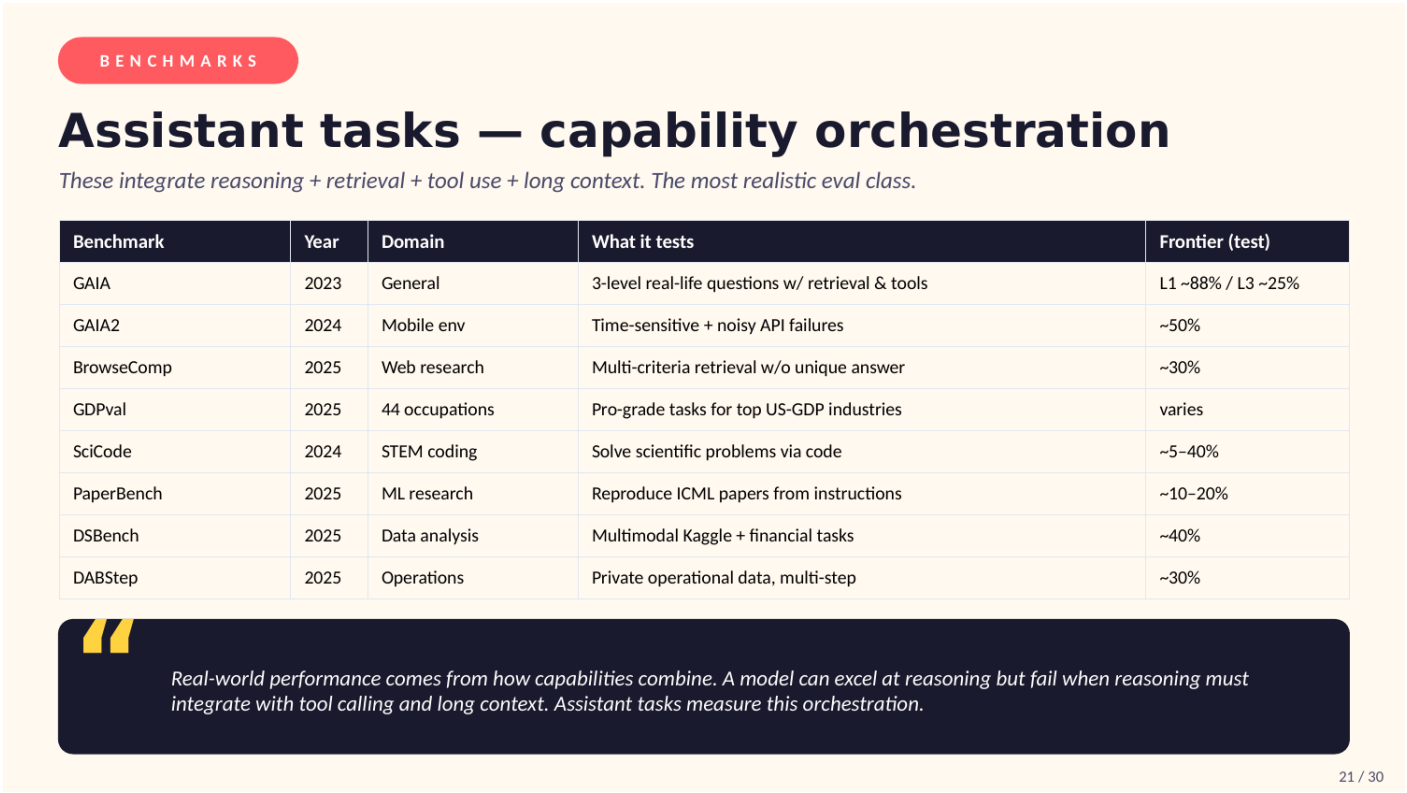
Tool calling & MCP

Tool-calling evaluation evolved from single API calls (BFCL v1–3) to multi-step real-world MCP server interactions (BFCL v4, MCP-Universe). The mock-versus-live tradeoff matters: mocked APIs give reproducibility but miss real-world failure modes; live APIs introduce non-determinism and rate-limiting.

- **TauBench (2024, ~55%):** mocked databases, multi-turn customer operations (retail/airline). Evaluated via DB state plus LLM judge.
- **BFCL v3 (2025, ~80%):** mocked APIs. Single-turn and multi-turn tool calls. Evaluated via Abstract Syntax Tree validation plus execution checks.
- **BFCL v4 (2025, ~62%):** adds web search and memory. Multi-criteria evaluation. The drop from v3 to v4 shows the cost of moving from controlled mocks to agentic tool use.
- **StableToolBench (2025, ~75%):** reliable mockup of the ToolBench environment. LLM judge.
- **MCPBench (2025, ~65%):** real Wikipedia/HF/Reddit MCP servers. Rules plus LLM judge.
- **MCP-Universe (2025, ~50%):** 11 MCP servers spanning 3D design, web search, navigation. Format plus execution evaluation.
- **LiveMCPBench (2025, ~80%):** tool selection from very long lists. Strict evaluation.

When to use which

BFCL is best for testing single-turn correctness because AST validation is deterministic. TauBench is realistic for multi-turn customer operations. MCP-Universe is best for measuring agentic tool selection in real environments. For KHCC AIDI applications, the right combo is BFCL v4 for unit-style coverage plus a custom MCP-Universe-style test against your internal MCP servers.



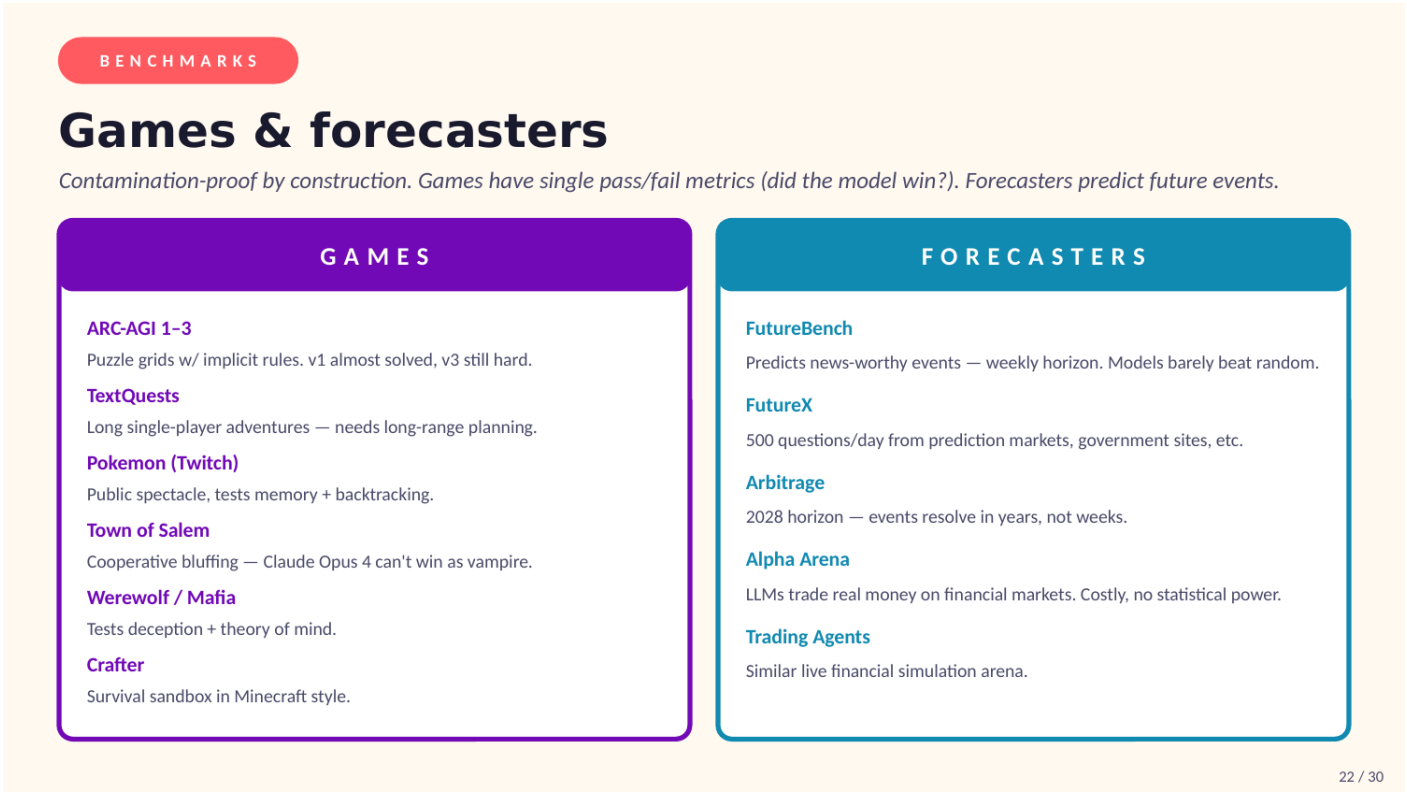
SLIDE 21

Assistant tasks — capability orchestration

Assistant-task benchmarks integrate reasoning, retrieval, tool use, and long context. They're the most realistic class of evaluation because real-world performance comes from how capabilities combine, not from any one capability in isolation.

- **GAIA (2023):** three difficulty levels of real-life questions with retrieval and tools. L1 around 88%; L3 around 25%.
- **GAIA2 (2024, ~50%):** mobile environment, time-sensitive queries, deliberately noisy API failures. Tests robustness to tool failures.
- **BrowseComp (2025, ~30%):** multi-criteria web research where the answer isn't unique — the model must find one valid answer matching multiple constraints.
- **GDPval (2025, varies):** professional-grade tasks across 44 occupations representing top US-GDP industries. Compared via model judges to human professional output.
- **SciCode (2024, ~5–40%):** solve real scientific problems by writing code. Decomposed into sub-problems for partial credit.
- **PaperBench (2025, ~10–20%):** reproduce ICML papers from instructions. Original authors contribute graded rubric trees.
- **DSBench (2025, ~40%):** multimodal Kaggle and financial-data tasks.
- **DABStep (2025, ~30%):** private operational data analysis, multi-step. Excellent because data was previously private (no contamination) and tasks have ground truth (no judge bias).

Real-world performance comes from how capabilities combine. A model can excel at reasoning but fail when reasoning must integrate with tool calling and long context. Assistant tasks measure this orchestration.



SLIDE 22

Games & forecasters

Games and forecasters are interesting evaluation classes because they're contamination-proof by construction. Games have a single unambiguous pass/fail metric — did the model win? Forecasters predict events that haven't happened yet.

Games

- **ARC-AGI 1–3:** puzzle grids with implicit rules. v1 almost solved, v3 still hard. Reminiscent of logic-oriented IQ tests.
- **TextQuests:** long single-player adventures — needs long-range planning, memory, and backtracking.
- **Pokemon (Twitch):** public spectacle running Claude and Gemini. Tests memory and backtracking abilities.
- **Town of Salem:** cooperative bluffing game. Claude Opus 4 can win as a peasant but cannot win as a vampire — fascinating asymmetry between cooperative and deceptive roles.
- **Werewolf / Mafia:** tests deception and theory of mind.
- **Crafter:** survival sandbox in Minecraft style.

Forecasters

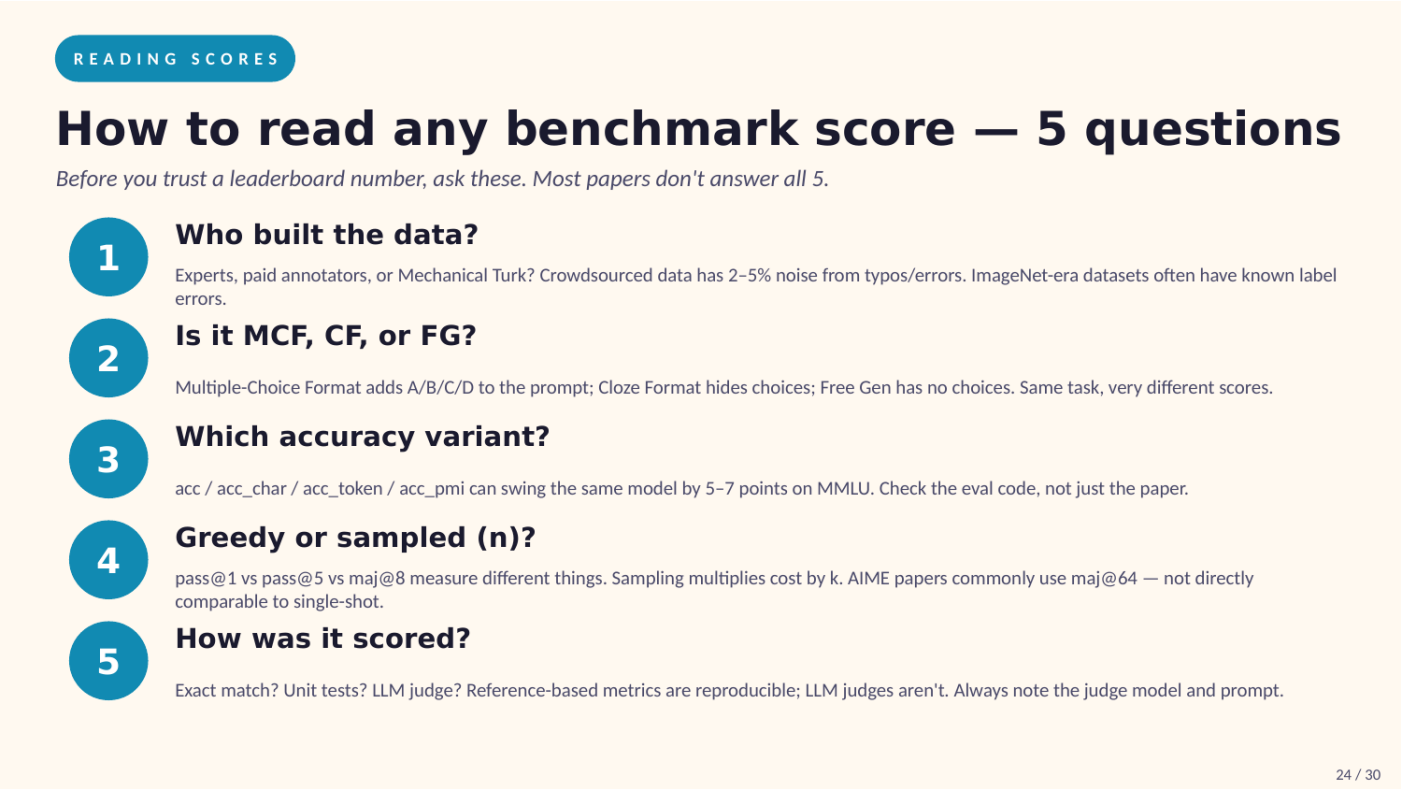
- **FutureBench:** predicts news-worthy events with weekly horizon. Models barely beat random.
- **FutureX:** 500 questions per day from prediction markets, government sites, ranking websites.
- **Arbitrage:** events resolve in 2028 — years rather than weeks.
- **Alpha Arena, Trading Agents:** LLMs trade real money on financial markets. Costly, no statistical power because each model is tested only once. Treat forecasting scores cautiously — it's hard to know whether near-random performance reflects genuine event unpredictability or model weakness.

SLIDE 23

Part 3 — Reading & Reproducing Scores

Part 3 is the most actionable section of the deck. It covers what to ask before trusting any benchmark number, why exact reproduction is harder than it looks, and what to demand alongside the headline score.

The premise is simple: a single number on a leaderboard is barely a number. Without prompt format, normalization scheme, sampling parameters, and code-base details, you cannot meaningfully compare two reported scores — even on the same benchmark.



SLIDE 24

How to read any benchmark score — 5 questions

Before trusting any leaderboard number, ask these five questions. Most papers don't answer all five. When they don't, treat the number as preliminary.

- **1. Who built the data?** Experts, paid annotators, or Mechanical Turk? Crowdsourced data has 2–5% noise from typos and errors. ImageNet-era datasets often contain known label mistakes.
- **2. Is it MCF, CF, or FG?** Multiple Choice Format adds A/B/C/D to the prompt; Cloze Format hides choices; Free Generation has no choices. Same task, very different scores. A 1.7B model might score 25% on MMLU in MCF but 50% in CF until it's trained enough to handle the multiple-choice convention.
- **3. Which accuracy variant?** acc, acc_char, acc_token, or acc_pmi. These can swing the same model by 5–7 points on MMLU. Check the eval code, not just the paper.
- **4. Greedy or sampled (n)?** pass@1 vs pass@5 vs maj@8 measure different things. Sampling multiplies cost by k. AIME papers commonly use maj@64 — not directly comparable to single-shot scores.
- **5. How was it scored?** Exact match? Unit tests? LLM judge? Reference-based metrics are reproducible across labs; LLM judges aren't, because the judge model and prompt differ. Always note the judge model, prompt, and version.

If you're a reviewer, ask for these. If you're publishing, provide them. If you're choosing a model based on a leaderboard, recognize you're often comparing apples to oranges.

SLIDE 25

Why you can't reproduce that paper's score

Even with the same code base, several factors silently shift results by a few points. Knowing them helps you both interpret reported numbers and avoid wasted hours chasing reproduction.

- **Different code base:** Different MMLU implementations across lm_eval, helm, and authors give different scores even for the same model. Default library settings vary.
- **Random seeds:** CUDA non-determinism plus sampling plus few-shot ordering can shift scores by ±2 points easily.
- **Prompt format:** “A. ...” vs “(A) ...” vs “Choices: A. ...” produces 7+ point swings on the same model.
- **Normalization:** Punctuation strip, casing, math answer extraction. Math-Verify versus naive SymPy makes a huge difference for math evaluations — the same prediction can score 0 or 1 depending on the parser.
- **Chat template / EOS:** Wrong system prompt or missing reasoning-trace removal causes model output to collapse into nonsense.
- **Hardware + library:** Different GPU, different transformers vs vLLM vs sglang — batching changes results subtly. PyTorch doesn't guarantee determinism across hardware.

Rule of thumb

If you reproduce within ±2 points of a paper using the same code base — success. If you're off by more than 5 points, you're running a different evaluation, even if it has the same name. Don't blame yourself; check the prompt format and normalization first.

SLIDE 26

What to report (and what to demand)

Beyond the raw score, evaluations should report enough to enable reproduction and meaningful comparison.

Always report

- **Score ± standard deviation.** Bootstrap or run at least 3 seeds. A single number is not science.
- **Total output tokens.** Reasoning models eat tokens — cost matters and so does latency.
- **Inference time / latency.** Especially for agentic and tool-use tasks where timing changes the user experience.
- **Sampling parameters.** Temperature, top-p, k for pass@k. All affect reproducibility.

2025 picks by use case

Pretraining ablations: ARC-Challenge, HellaSwag (CF), MMLU, GSM8K — not because they're best, but because they're well-understood and provide signal early in training.

Post-training (compare frontier models): GPQA Diamond, MMLU-Pro, IFEval, AIME 2025, MathArena, AiderBench, LiveCodeBench, HLE.

Tool-using agents: BFCL v4, MCP-Universe, TauBench.

Real-world assistant tasks: GAIA2, DABStep, SciCode, plus a custom domain-specific evaluation built from your own data.

Long context: HELMET, Novel Challenge, Kalamang Translate.

Pick benchmarks that match your goals. Run several rather than one. Always pair a public benchmark with a custom evaluation for your specific use case.

SLIDE 27

Part 4 — Build Your Own Eval

Part 4 covers the practitioner workflow: when public benchmarks don't fit your domain, how do you actually build evaluations that catch problems before users do?

The Data Lumina training video introduces three levels of evaluation. **Level 1:** unit tests — fast, cheap boolean checks run on every commit. **Level 2:** human plus LLM-as-judge — systematic review of quality, run weekly or bi-weekly. **Level 3:** A/B testing — real users vote, run with major releases.

This handout covers Level 1 and Level 2 in depth. Level 3 is application-specific and only worthwhile when your user base is large enough to provide statistical power.

SLIDE 28

Level 1: scope-specific unit tests

Unit tests for LLMs are exactly what they sound like. You write Python assertions against the structured output of your AI workflow, run them on every commit, and refuse to ship if any fail. They are cheap, fast, and catch about 80% of regressions before users do.

The example on the slide tests a customer-service classifier. We load a real billing event from JSON, run our classify() function, and assert four things: the category equals “billing”; the confidence is between 0 and 1; the reply is more than 10 characters; and the reply mentions “refund.” Each assertion encodes a piece of business logic the system must respect.

Principles

- **Capture real events.** Save every production event as JSON in evals/events/. The events folder IS your evaluation suite. Nothing else. This makes evaluation grounded in actual user behavior, not in synthetic test cases.
- **Build from failures.** Every production incident becomes a new test case. Once you've fixed a bug, write a test that catches it. Never lose ground twice.
- **Use pytest for scale.** When you have 50+ tests, switch from a hand-rolled runner to pytest. You get better reporting, parallel execution, and fixtures for shared setup.
- **Run per pipeline step.** Don't just test the final output. If your workflow has a classifier, then a router, then a response generator, assert at each step. This localizes failures and makes debugging fast.

Treat your eval suite the way you treat your code: version-controlled, reviewed, expanded over time. The KHCC AIDI production pipelines all follow this pattern — every pipeline ships with a folder of real events and a script that runs them through the workflow with assertions.

SLIDE 29

Level 2: LLM-as-judge — the alignment loop

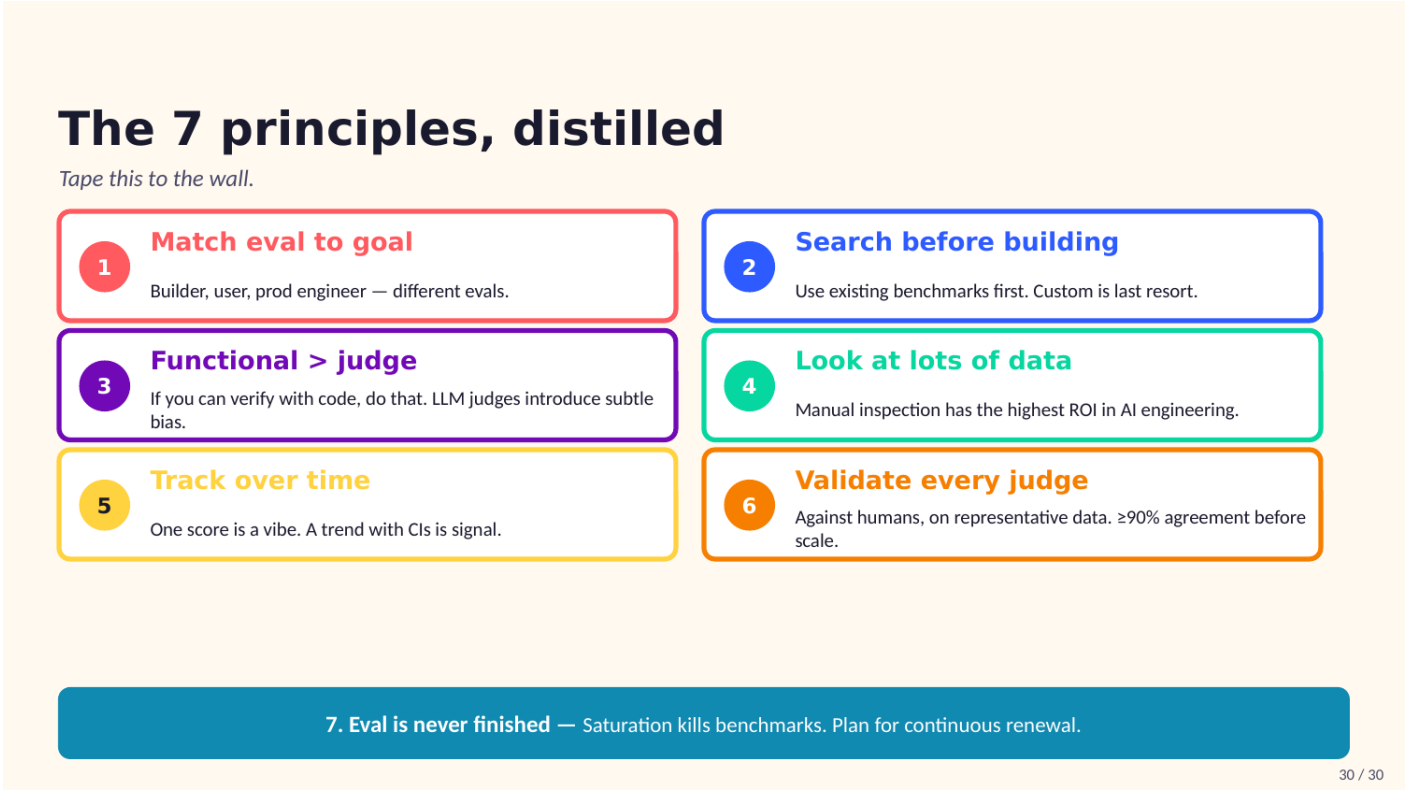
Once unit tests cover the deterministic behavior, you still need to evaluate qualitative properties: tone, factuality, helpfulness. LLM-as-judge means using a powerful model to grade your application's outputs against a rubric you define. Done right, it scales human evaluation to thousands of samples. Done wrong, it's an echo chamber that reinforces whatever bias the judge happens to have.

The 5-step alignment loop

- **1. Sample 50–100.** Pick representative real events that cover both happy paths and edge cases. Quality matters more than quantity.
- **2. Human grades.** You or a domain expert reads each output and labels it good/bad with a one-sentence reason. This is your golden standard.
- **3. Judge grades.** Use a powerful model with a structured-output grading prompt to label the same outputs.
- **4. Compute agreement.** Confusion matrix → percentage agreement between human and judge. Target at least 90%.
- **5. Iterate prompt.** Use disagreements to refine the judge prompt. The disagreements tell you exactly where the judge is missing the rubric. Repeat until agreement plateaus.

Known judge biases to watch for

Position bias (likes the first answer in a comparison). Verbosity bias (longer means better). Self-preference (favors outputs from its own model family). Format bias (prefers markdown over plain text). No internal consistency at temperature greater than 0. Diverges from humans specifically on hallucination detection — judges are notoriously bad at catching subtle factual errors. **Never deploy a judge that hasn't been validated against humans** — an unvalidated judge is just a confident-sounding random number generator.



SLIDE 30

The 7 principles, distilled

These seven principles summarize the entire deck. Print them, tape them to the wall, and refer back when you're tempted to skip a step.

- **1. Match eval to goal.** Builder, user, and prod engineer need different evals. Don't use a research benchmark to validate a production change.
- **2. Search before building.** Use existing benchmarks first; build custom only when nothing fits. Custom evals are expensive to maintain.
- **3. Functional > judge.** If you can verify with code, do it. LLM judges introduce subtle bias and aren't reproducible across labs.
- **4. Look at lots of data.** Manual inspection has the highest ROI in AI engineering. The Zapier principal engineer who spends 3 hours per day looking at traces is doing the right thing.
- **5. Track over time.** One score is a vibe. A trend with confidence intervals is signal. Plot scores per checkpoint, per release, per week.
- **6. Validate every judge.** Against humans, on representative data. Aim for at least 90% agreement before scaling. An unvalidated judge is worse than no judge.
- **7. Eval is never finished.** Saturation kills benchmarks. Plan for continuous renewal — retire saturated tests, add new failure modes, refresh data.

Closing

Evaluation is the highest-leverage activity in AI engineering. It catches regressions, surfaces hidden capabilities, and drives the iteration loop that turns demos into reliable systems. The teams that win are the teams that take evaluation seriously — not as a gate at the end, but as a daily practice.